WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Timing

Lecture 09: propagation delay, contamination delay and glitches

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Time in circuits

02 Propagation and contamination delay

03 Glitches

04 Why timing matters

05 Summary

01

Time in circuits

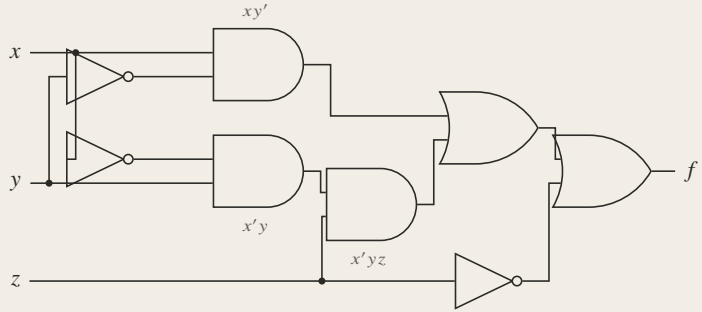
On paper, there is no time

Take $f(x, y, z) = xy' + x'yz + z'$.

- Computing $f(1, 1, 0)$ by hand might take you 5 seconds. But you only give the answer when you are done, so whoever hears it knows it is right.
- A circuit cannot keep quiet until it is done. Its output wire always carries *some* value, 0 or 1, even while its gates are still working.

> **The question for today**
>
> After the inputs change, how long until the output can be trusted? And what does it do in the meantime?

The example circuit



- 0 and 1 are voltages, for example 0 V and 5 V. Keep the inputs at $x, y, z = 1, 1, 0$ and the output settles at $f = 1$.
- Then change the inputs to $0, 1, 1$. Here too $f(0, 1, 1) = 1$, so we expect the output to stay at 1. Does it?

Delays

- A signal needs time to travel through wires and through the transistors inside each gate: a delay. The longer the wire, the longer the delay.
- To keep things simple, as the book does: every gate takes exactly 1 ns, and wires take no time.
- So a gate's output at time t is what its inputs were at time $t - 1\,\text{ns}$.

> **Remark**
>
> Real gates switch in far less than a nanosecond, and different gates have different delays. The method is the same: only the numbers change.

02

Propagation and contamination delay

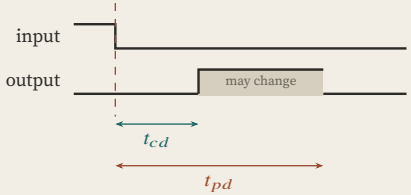
Two delays

Definition (Propagation delay t_{pd})

The *longest* time from a change at the inputs until the outputs have reached their final values.

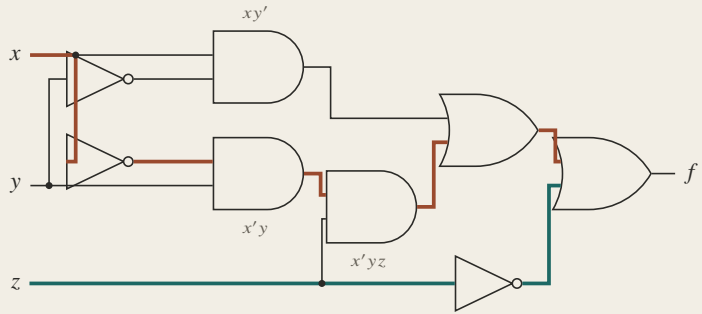
Definition (Contamination delay t_{cd})

The *shortest* time from a change at the inputs until an output starts to change.



Before t_{cd} the output still shows the old answer; after t_{pd} it shows the new one. In between, it may do anything.

Finding the delays: paths



- Longest path, the critical path: $x \to$ NOT $\to$ AND $\to$ AND $\to$ OR $\to$ OR, 5 gates, so $t_{pd} = 5$ ns.
- Shortest path: $z \to$ NOT $\to$ OR, 2 gates, so $t_{cd} = 2$ ns.
- The other four paths from an input to f have 3 or 4 gates.

Gates with different delays

- Each gate has its own t_{pd} and t_{cd}, with $t_{cd} \leq t_{pd}$.
- The circuit's t_{pd} is the largest sum of gate t_{pd}'s along any path from an input to an output.
- Its t_{cd} is the smallest sum of gate t_{cd}'s along any path.

> **Warning**
>
> With different gate delays, the critical path is not always the path with the most gates. Add up the delays of every path, or at least of every path that could be the longest.

03

Glitches

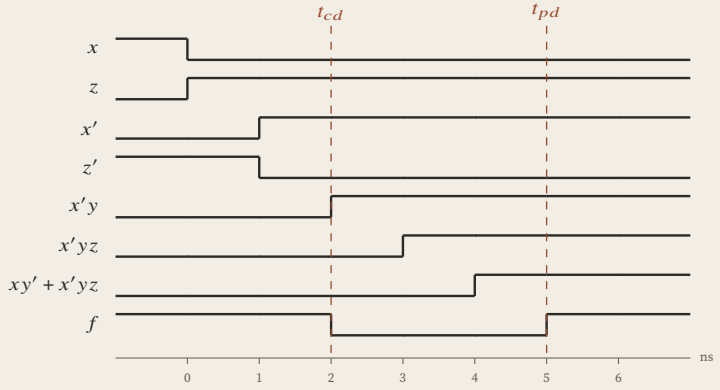
Watching the change, nanosecond by nanosecond

The inputs change from $x, y, z = 1, 1, 0$ to $0, 1, 1$ at $t = 0$.

t (ns)	before	0	1	2	3	4	5	6
x'	0	0	1	1	1	1	1	1
z'	1	1	0	0	0	0	0	0
$x'y$	0	0	0	1	1	1	1	1
$x'yz$	0	0	0	0	1	1	1	1
$xy' + x'yz$	0	0	0	0	0	1	1	1
f	1	1	1	0	0	0	1	1

- z' falls after 1 ns, so f falls after 2 ns: nothing else is 1 yet.
- The term $x'yz$ needs 4 ns to rise, so f is back at 1 only after 5 ns.

The timing diagram



y stays 1 throughout. Time 0 is when x and z change.

Glitches

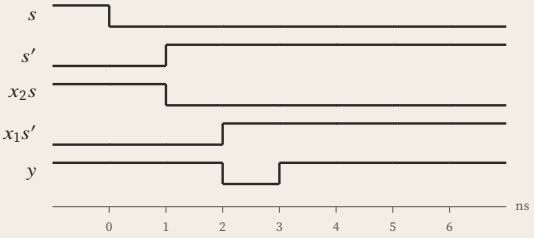
> **Definition (Glitch)**
>
> A *glitch* is a short wrong value at an output while the circuit settles after its inputs change.

- Here f should stay at 1, but it is 0 from 2 ns to 5 ns.
- Anyone who reads f in that window gets the wrong answer.
- After t_{pd} the output is always right. So the simplest rule is: wait at least t_{pd} after the inputs change, then read the output.

This needs t_{pd} to be known, which is why we compute it.

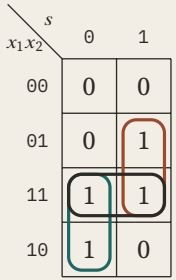
A glitch from a single input

The multiplexer of Lecture 07, $y = x_1 s' + x_2 s$, with $x_1 = x_2 = 1$. Then $y = 1$ whatever s is. Now switch s from 1 to 0:



- $x_2 s$ turns off after 1 ns.
- $x_1 s'$ waits for the NOT gate, and turns on only after 2 ns.
- For 1 ns neither door is open: y glitches to 0.

Removing the glitch



- The change goes from cell 111 to cell 110: two 1-cells next to each other but in different groups.
- Add the group that covers both, x_1x_2: the consensus term we left out in Lecture 07.
- $y = x_1s' + x_2s + x_1x_2$ is the same function, and x_1x_2 stays 1 while s changes: no glitch.

> **Remark**
>
> The price is one more gate. Usually we do not pay it: we wait for t_{pd} instead, and a clock tells every part of the computer when to read `(Unit 11–14)`.

04

Why timing matters

The speed limit

- A result can only be used t_{pd} after the inputs change. With $t_{pd} = 5\,\text{ns}$, that is at most one result every 5 ns:

$$\frac{1}{5\,\text{ns}} = 200 \text{ million results per second.}$$

- To make a circuit faster, shorten its critical path.
- In sequential circuits both delays matter: t_{pd} sets how fast the clock can tick, and t_{cd} how soon a stored value may be disturbed. These are the setup and hold time constraints (Unit 14).

The book: delays become more and more important as we build a real computer.

Try it

1. In the example circuit, let NOT take 1 ns, and AND and OR 2 ns each. What are t_{pd} and t_{cd}?

2. With 1 ns gates, does the multiplexer glitch when s goes from 0 to 1?

3. Why does a longer wire have a longer delay?

Try it

1. In the example circuit, let NOT take 1 ns, and AND and OR 2 ns each. What are t_{pd} and t_{cd}?
   $t_{pd} = 1 + 2 + 2 + 2 + 2 = 9$ ns, still along $x \to$ NOT $\to$ AND $\to$ AND $\to$ OR $\to$ OR. $t_{cd} = 1 + 2 = 3$ ns, along $z \to$ NOT $\to$ OR.
2. With 1 ns gates, does the multiplexer glitch when s goes from 0 to 1?

3. Why does a longer wire have a longer delay?

Try it

1. In the example circuit, let NOT take 1 ns, and AND and OR 2 ns each. What are t_{pd} and t_{cd}?
   $t_{pd} = 1 + 2 + 2 + 2 + 2 = 9$ ns, still along $x \to$ NOT $\to$ AND $\to$ AND $\to$ OR $\to$ OR. $t_{cd} = 1 + 2 = 3$ ns, along $z \to$ NOT $\to$ OR.
2. With 1 ns gates, does the multiplexer glitch when s goes from 0 to 1?
   No. $x_2 s$ turns on after 1 ns, and $x_1 s'$ turns off only after 2 ns: for 1 ns both are 1, so y stays 1.
3. Why does a longer wire have a longer delay?

Try it

1. In the example circuit, let NOT take 1 ns, and AND and OR 2 ns each. What are t_{pd} and t_{cd}?
   $t_{pd} = 1 + 2 + 2 + 2 + 2 = 9$ ns, still along $x \to$ NOT $\to$ AND $\to$ AND $\to$ OR $\to$ OR. $t_{cd} = 1 + 2 = 3$ ns, along $z \to$ NOT $\to$ OR.
2. With 1 ns gates, does the multiplexer glitch when s goes from 0 to 1?
   No. $x_2 s$ turns on after 1 ns, and $x_1 s'$ turns off only after 2 ns: for 1 ns both are 1, so y stays 1.
3. Why does a longer wire have a longer delay?
   The signal takes time to travel, and a longer wire takes longer to change its voltage.

05

Summary

Summary

- Real circuits take time: every gate and wire adds a delay, and the output always shows some value, right or not.
- Propagation delay t_{pd}: the longest path, the time until the output is final. Contamination delay t_{cd}: the shortest path, the time until it can first change.
- A glitch is a short wrong output while the circuit settles. Reading after t_{pd} is always safe.
- A glitch from one input can be removed with the consensus term, at the cost of a gate.
- t_{pd} limits how fast a circuit can be used.

Before the next lecture

- Read Section 4.2 of the book, *Timing*.
- For $g = xy + x'z$, list every path, and find t_{pd} and t_{cd} with 1 ns gates. Which single input change can make g glitch, and how do you remove it?
- In Logisim-evolution, build the example circuit as in the book: one input *ctrl* drives x directly and z through a NOT gate, so both change together. Switch *ctrl* from 1 to 0 and find the glitch in the timing diagram.

> **Next lecture** (Unit 10)
>
> Sequential logic: finite state machines, Moore and Mealy machines.